

Rustを始めよう！ついでに競プロも始めよう！

藤森 大地

本勉強会の目指すところ

- Rust面白い！と感じてもらう
- Rustを始める第一歩としてもらえたら

本日の流れ

- 解説
- 問題とく

Rustとはどんな言語か



- 静的型付け言語
- コンパイル言語

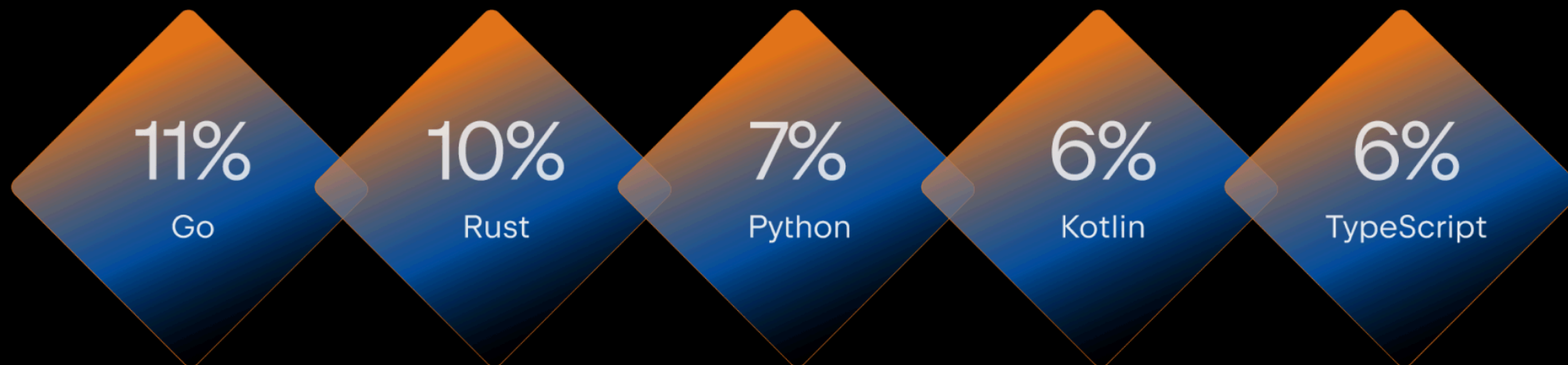
メモリ安全性を重視

Rustへの評価

開発者が次に採用したいと考えている上位5つのプログラミング言語に長年入っている

TOP FIVE LANGUAGES DEVELOPERS WANT TO ADOPT NEXT

Share of developers expressing an intention to adopt this language



今日の題材について

競技プログラミング(AtCoder)ってなに？

競技プログラミングについて

決められた問題を、制限時間内にプログラムで解く競技。
下記のような問題が出る。

整数 N が与えられる。

1 から N までの合計を求めよ。

競技プログラミングについて

何を競うのか

- 制限時間内に実行できるか
- メモリ制限内に収まるか
- どれだけ早く解けるか

これが出来ると何がいいのか

論理的思考、数学的な知識、計算量への意識、言語への理解が深まる

競技プログラミング有名所



AtCoder



CODEFORCES

AtCoderについて

日本語向け競技プログラミングサービスの定番
わかりやすい回答解説も豊富

本日は、、、

AtCoderの問題を通してRustを学ぶぞ。

以下早速はじめて行こう！

事前準備

- Rustのインストール
- VSCode + Rust拡張のインストール
- AtCoderのアカウント作成

Hello World

```
cargo init project_name
```

```
fn main() {  
    println!("Hello, world!");  
}
```

```
cargo run
```

変数宣言

```
let x = 5; // 不変な変数
let mut y = 10; // 可変な変数
println!("x:{} y:{}", x, y);

x += 1;
y += 1;
println!("x:{} y:{}", x, y); // フォーマットして出力するprintの書き方
```

変数宣言

```
error[E0384]: cannot assign twice to immutable variable `x`  
--> src/main.rs:6:5
```

```
2 |     let x = 5; // 不変な変数  
  |           - first assignment to `x`  
...  
6 |     x += 1;  
  |     ^^^^^ cannot assign twice to immutable variable
```

```
help: consider making this binding mutable
```

```
2 |     let mut x = 5; // 不変な変数  
  |           +++
```

For more information about this error, try `rustc --explain E0384`.
error: could not compile `demo\_pro` (bin "demo_pro") due to 1 previous error

型について

静的型付け/動的型付け言語について説明。

i32, String型について説明

Rustの基本の書き方

- 静的言語(プリミティブ？型について)
 - i32, f32, u32, usize
 - String, str, chr
- 変数宣言
- for
- if
- 標準入力

if文

```
let n = 10;

if n % 2 == 0 {
    println!("nは偶数");
} else {
    println!("nは奇数");
}
```

for文

```
let arr = [1, 2, 3, 4, 5]; // 配列(固定長)

for i in 0..5 {
    println!("{}", arr[i]);
}
```

```
cargo add proconio
```

```
use proconio::input;
fn main() {
    input! {
        n: usize,
        a_list: [i32; n],
    }

    for a in a_list {
        println!("{}", a);
    }
}
```

```
5
1 2 3 4 5
```

```
cargo run < input.txt
```

問題を解いてみよう！

mut, if, for

AtCoder ABCコンテスト 330 A問題

https://atcoder.jp/contests/abc330/tasks/abc330_a

A - Counting Passes

解説

 / 

実行時間制限: 2 sec / メモリ制限: 1024 MiB

配点: 100 点

問題文

N 人の人 $1, 2, \dots, N$ がある試験を受け、人 i は A_i 点を取りました。

この試験では、 L 点以上を取った人のみが合格となります。

N 人のうち何人が合格したか求めてください。

制約

- 入力は全て整数
- $1 \leq N \leq 100$

回答

```
let mut count = 0;

for i in 0..n {
    if a_list[i] >= l {
        count += 1;
    }
}

println!("{}", count);
```

別解

```
for a in &a_list {  
    if a >= l {  
        count += 1;  
    }  
}
```

速さについて多言語と比較

どの問題にするのか未決定

候補1 abc051_b

- 3重だとTLE(Rust)
- 2重だと他の言語でもAC

○候補2 abc162_c

- 一応コードやオーダー的には3重
- PythonのみTLE (Rust, PHP, Python, TS, JS)

所有権・借用について

代入と束縛(バインド)について

(図を入れる)

所有権とは
値を管理しているのが誰かを示す

コードで説明

```
let s = String::from("Hello, world!"); // "Hello, world!".to_string()でもOK
println!("{}", s);
```

```
let s2 = s; // sの所有権がs2に移動する
```

```
println!("{}", s2);
```

```
// println!("{}", s); // エラー: sはs2に所有権が移動しているため、sは使用できない
```

同じ値を何回も使いたいが、所有権を消費してしまっているのでError（困った）

```
fn print_string(s: String) {  
    println!("{}", s);  
}  
  
fn main() {  
    let s = String::from("Hello, world!"); // "Hello, world!".to_string()でもOK  
  
    print_string(s);  
    // print_string(s); // エラー: sはprint_stringに所有権が移動しているため、sは使用できない  
}
```

リファレンス(参照)を借用する

```
fn print_string_ref(s: &str) {  
    println!("{}", s);  
}  
  
fn main() {  
    let s = String::from("Hello, world!"); // "Hello, world!".to_string()でもOK  
  
    print_string_ref(&s);  
    print_string_ref(&s);  
}
```

(図 p.62)

問題を解いてみよう！(2)

所有権, 借用, String

AtCoder ABCコンテスト 149 A問題

https://atcoder.jp/contests/abc149/tasks/abc149_a

AtCoder ABCコンテスト 103 B問題

https://atcoder.jp/contests/abc103/tasks/abc103_b

A - Strings

解説

 / 

実行時間制限: 2 sec / メモリ制限: 1024 MiB

配点: 100 点

問題文

英小文字のみからなる2つの文字列 S, T が与えられます。これらの文字列を T, S の順に空白を空けずに連結し、できる文字列を出力してください。

制約

S, T は英小文字のみからなる文字列

149Aの回答解説

```
println!("{}", t, s);
```

```
let ans = t + &s;  
println!("{}", ans);
```

```
let ans = t + &s;  
println!("{}", ans);
```

```
println!("{}", s);  
println!("{}", t); // 所有権を持っていないのでErrorとなる
```

Rustの文字列の2項演算について

`Strign = String + &str`

所有権が動く

103Bについて

B - String Rotation

[解説](#)[🇯🇵](#) / [🇬🇧](#)

実行時間制限: 2 sec / メモリ制限: 1024 MiB

配点 : 200 点

問題文

英小文字からなる文字列 S, T が与えられます。

S を回転させて T に一致させられるか判定してください。

すなわち、以下の操作を任意の回数繰り返して S を T に一致させられるか判定してください。

操作: $S = S_1 S_2 \dots S_{|S|}$ のとき、 S を $S_{|S|} S_1 S_2 \dots S_{|S|-1}$ に変更する

ここで、 $|X|$ は文字列 X の長さを表します。

制約

- $2 \leq |S| \leq 100$
- $|S| = |T|$
- S, T は英小文字からなる

103Bの回答解説

// sをコピーしないと所有権が移ってしまい参照が使えなくなってしまう。

```
let ss = s.clone() + &s;
```

```
if ss.contains(&t) {  
    println!("Yes");  
} else {  
    println!("No");  
}
```

まとめ

所有権を中心に学びながら

Rustで問題を解いてみた

楽しいと思ってもらえたら

AtCoderの成績が全然振るわないので仲間増えたら嬉しいです！